

# Chapter 21 - MIDI/MUS and RawlandMini GM Synth

---

The MIDI/MUS player is the bridge between file music and the IE-native mixer. It accepts Standard MIDI Files and Doom MUS data, turns their notes, programmes, controllers, tempo changes, and pitch bends into events, then renders them with the built-in RawlandMini patch table.

This is not a separate historical sound chip. It is an Intuition Engine synth player on the shared audio bus. Use it when you want general song playback with melodic programmes and a drum channel, but you do not want to write note events one register at a time.

## 21.1 First SMF sound

---

Type this program. It builds a tiny Standard MIDI File in memory, starts it with the raw MIDI registers, and leaves it looping long enough to inspect the status and tempo registers.

```
10 REM TINY RAWLANDMINI MIDI CHORD
20 POKE &H000F0800,1
30 A=&H00120000:L=58
40 FOR I=0 TO L-1
50 READ B
60 POKE8 A+I,B
70 NEXT I
80 POKE8 &H000F0BB4,220
90 POKE &H000F0BA0,A
100 POKE &H000F0BA4,L
110 POKE &H000F0BA8,5
120 FOR T=1 TO 3000
130 NEXT T
140 PRINT PEEK(&H000F0BAC),PEEK(&H000F0BB8)
150 POKE &H000F0BA8,2
200 DATA 77,84,104,100,0,0,0,6,0,0,0,1,0,96
210 DATA 77,84,114,107,0,0,0,36
220 DATA 0,192,80,0,176,7,100
230 DATA 0,144,60,100,0,144,64,100,0,144,67,100
240 DATA 129,64,128,60,0,0,128,64,0,0,128,67,0
250 DATA 0,255,47,0
```

You should hear a short three-note chord. Line 140 normally prints 1 120: status bit 0 means the player is busy or playing, and the tempo register reports 120 BPM. If you read the status register immediately after line 110, bit 3 may also be set while the player is still parsing the file.

Lines 40 to 70 copy the file bytes into ordinary RAM. Lines 80 to 110 set the global MIDI volume, stage the pointer and length, then start with looping enabled. The first DATA line is the MThd header for an SMF type 0 file with one track and division 96. The second line is the MTrk header. The remaining bytes select programme 80, set channel volume, play C, E, and G, release them after two quarter notes, and end the track.

Try changing line 80 to a smaller value such as 90. The same notes play with the same patch table, but the whole MIDI player is quieter.

## 21.2 First Doom MUS sound

Doom MUS uses a smaller event stream. This example builds the shortest useful MUS block: one note-on event, one note-off event, and an end event. It uses the same player registers as the SMF example.

```
10 REM TINY DOOM MUS BLIP
20 POKE &H000F0800,1
30 A=&H00120100:L=24
40 FOR I=0 TO L-1
50 READ B
60 POKE8 A+I,B
70 NEXT I
80 POKE8 &H000F0BB4,200
90 POKE &H000F0BA0,A
100 POKE &H000F0BA4,L
110 POKE &H000F0BA8,5
120 FOR T=1 TO 2000
130 NEXT T
140 PRINT PEEK(&H000F0BAC),PEEK(&H000F0BB8)
150 POKE &H000F0BA8,2
200 DATA 77,85,83,26,8,0,16,0,0,0,0,0
210 DATA 0,0,0,0,144,188,100,70,128,60,1,96
```

Line 140 normally prints 1 140. The player is active, and the MUS timing model reports 140 BPM. The header starts with MUS followed by byte \$1A; the score length is 8 bytes and the score begins at offset 16.

The score byte \$90 means note-on, channel 0, and end-of-group. The note byte \$BC is note 60 with a new velocity byte following. The next event turns note 60 off, then the final \$60 marks the end of the MUS score.

## 21.3 What the player accepts

| Item            | Value                                            |
|-----------------|--------------------------------------------------|
| SMF formats     | Type 0 and type 1                                |
| SMF timing      | Ticks per quarter note                           |
| SMF tempo       | Tempo meta-events, default 120 BPM               |
| SMF title       | First track-name meta-event                      |
| SMF unsupported | Type 2, SMPTE division                           |
| MUS format      | Doom MUS header MUS plus byte \$1A               |
| MUS timing      | 140 ticks per second                             |
| Patch table     | Built-in RawLandMini                             |
| Output voices   | Up to 10 active voices                           |
| Output path     | IE audio mixer, through the global effects chain |

Recognised SMF channel events are note on, note off, programme change, channel volume controller 7, expression controller 11, and pitch bend. Other controller and system messages are skipped unless they are needed to find track length or tempo.

SMF type 1 tracks are merged into one time-ordered event stream. Sysex events are skipped. A missing tempo starts at 120 BPM.

MUS channel 15 maps to MIDI channel 9, the drum channel. MUS channel 9 maps away to MIDI channel 15, so normal melodic MUS channel 9 does not accidentally become percussion.

## 21.4 Register block

The MIDI player register block is \$F0BA0-\$F0BBF.

| Address | Name             | Access     | Purpose                                       |
|---------|------------------|------------|-----------------------------------------------|
| \$F0BA0 | MIDI_PLAY_PTR    | write/read | Low 32 bits of the SMF or MUS block address.  |
| \$F0BA4 | MIDI_PLAY_LEN    | write/read | Length of the SMF or MUS block, in bytes.     |
| \$F0BA8 | MIDI_PLAY_CTRL   | write/read | Start, stop, loop, pause, and resume control. |
| \$F0BAC | MIDI_PLAY_STATUS | read       | Busy, error, pause, and loading status bits.  |
| \$F0BB0 | MIDI_POSITION    | read       | Current playback position in output samples.  |
| \$F0BB4 | MIDI_VOLUME      | write/read | Global MIDI volume, 0 - 255.                  |
| \$F0BB8 | MIDI_TEMPO_BPM   | read       | Current effective tempo in BPM.               |

The pointer and length are 32-bit low-window values. Put the bytes in ordinary readable memory, write pointer and length, then write the control register.

## 21.5 Control and status bits

Write these values to MIDI\_PLAY\_CTRL:

| Bit | Value | Meaning                                                        |
|-----|-------|----------------------------------------------------------------|
| 0   | 1     | Start playback using the staged pointer and length.            |
| 1   | 2     | Stop playback and clear the current start request.             |
| 2   | 4     | Loop when the end is reached. Combine with start as value 5.   |
| 3   | 8     | Pause without resetting the source position.                   |
| 4   | 16    | Apply the loop bit to the current playback without restarting. |

To resume after pause, write 0 to MIDI\_PLAY\_CTRL. Writing 1 starts again from the staged pointer and length.

Read MIDI\_PLAY\_CTRL for the current control state:

| Bit | Meaning                                |
|-----|----------------------------------------|
| 0   | Loading is busy or playback is active. |
| 2   | Loop is enabled.                       |
| 3   | Pause is enabled.                      |

Read MIDI\_PLAY\_STATUS for player state:

| Bit | Meaning                                                           |
|-----|-------------------------------------------------------------------|
| 0   | Busy or playing.                                                  |
| 1   | Last start request failed.                                        |
| 2   | Paused.                                                           |
| 3   | Loading: an asynchronous parse/load request is still in progress. |

The error bit is set when the block cannot be read or cannot be parsed as SMF or MUS. A successful new start clears the old error.

The busy bit is broad: it is set while the player is loading and also while a song is playing. The loading bit is narrower. It is set only for the parse/load step started by MIDI\_PLAY\_CTRL. Poll it if your program needs to know when the source bytes have been accepted:

```

10 REM WAIT FOR MIDI OR MUS LOAD TO FINISH
20 S=PEEK(&H000F0BAC)
30 IF (S AND 8)<>0 THEN GOTO 20
40 IF (S AND 2)<>0 THEN GOTO 70
50 PRINT "READY ";S
60 END
70 PRINT "BAD MIDI"

```

When the loop reaches line 50, bit 3 is clear. If bit 1 is set, the file was rejected and the old song, if any, should not be trusted as the result of the new start request.

## 21.6 Setup order

From a clean state:

1. Enable the audio mixer by writing 1 to \$F0800.
2. Put the SMF or MUS bytes in memory.
3. Write MIDI\_PLAY\_PTR and MIDI\_PLAY\_LEN.
4. Optionally write MIDI\_VOLUME.
5. Write 1 to MIDI\_PLAY\_CTRL, or 5 for start plus loop.
6. Poll MIDI\_PLAY\_STATUS bit 3 until it clears if the program must wait for parsing to finish.
7. Read MIDI\_PLAY\_STATUS and MIDI\_TEMPO\_BPM if you need proof.

The player parses the bytes when start is written. Changing the source memory after start does not alter the currently loaded song. Stop and start again if you want the player to read a changed block.

## 21.7 RawlandMini patch table

RawlandMini is the fixed synth table used by the MIDI engine. It has 128 melodic programme entries and a separate drum table. Melodic programmes are grouped in GM-style families: piano, chromatic percussion, organ, guitar, bass, strings, ensemble, brass, reed, pipe, synth lead, synth pad, synth effects, ethnic, percussive melodic, and sound effects.

Each RawlandMini patch chooses one IE waveform, ADSR-like timing, and a level. Channel volume, expression, velocity, and MIDI\_VOLUME then scale the final voice. Pitch bend covers two semitones either side of centre.

Channel 9 is the drum channel. Notes 35 through 81 use distinct drum-family patches for kicks, snares, toms, hats, cymbals, bells, and small percussion. Other drum notes still produce a default noise hit.

RawlandMini is built into Intuition Engine. There is no reader-facing soundfont loader in this register block.

## 21.8 Media loader path

---

The media loader recognises .mid, .midi, and .mus filenames. From BASIC, the shortest path is:

```
10 SOUND PLAY "SONG.MID"  
20 SOUND STOP
```

The same SOUND PLAY form also accepts Doom MUS files:

```
10 SOUND PLAY "E1M1.MUS"  
20 SOUND STOP
```

When the media loader selects the MIDI player, MEDIA\_TYPE reads 8. Chapter 23 lists the full loader register protocol and the filename extensions for every music engine.

## 21.9 IE Script path

---

IE Script exposes the same player for automated runs:

```
audio.midi_load("SONG.MID")  
audio.midi_play()  
audio.midi_set_volume(180)  
audio.midi_stop()
```

audio.midi\_metadata() returns a table with the title, system name, duration text, format name, track count, and patch-table name when a song has been loaded. Chapter 34 describes IE Script as an automation surface around the machine, not as the normal typed BASIC path.

## 21.10 Limits

---

- SMF type 2 is unsupported.
- SMPTE time division is unsupported.
- Only the event types listed in section 21.3 affect playback.
- The active voice budget is 10; when more voices are requested, the engine steals an existing voice using a deterministic priority rule.
- RawlandMini is fixed for this release. Programs can select GM-style programme numbers, but cannot replace the patch table.
- MIDI and MUS playback share the same global mixer effects as the other engines. Heavy overdrive, filtering, or reverb changes the whole mix, not only the MIDI player.

Chapter 22 covers Paula DMA for programs that need direct sample-buffer control. Chapter 23 shows how BASIC and the media loader choose among all music engines.